MODULE

12

Capstone: rescue a messy repo

Look, find, plan, act: fix any broken repository calmly and ship a release.

YOU WILL LEARN

Rescue walkthrough	`01_rescue_walkthrough.py`
Exercise: rescue + release	`ex12_rescue_the_repo.py`
Final exam	`QUIZ.md`
Interview Q&A	`INTERVIEW.md`

WHY RESCUE

”Help, I broke Git!”

THE PROBLEM

Committed to main by mistake. Pushed a debug file. A teammate pushed meanwhile. The release has to go out today. Typing random commands from Stack Overflow makes it worse.

THE IDEA

Use the same 4 steps every time: LOOK, FIND, PLAN, ACT. With a safety branch, every step is reversible.

PICTURE IT RESCUE

Look, find, plan, act

🩺 Like a doctor: check the symptoms, read the history, decide on the treatment, then treat one step at a time and check after each one.

LOOK
status -sb, log --graph --all, stash list

→

FIND
reflog, log origin/main..main

→

PLAN
what should each branch look like?

→

ACT
git branch backup, then small steps

TRY IT LOOK AND FIND

Diagnose the mess

```
$ git fetch -q
$ git status -sb
## main...origin/main [ahead 1, behind 1]
$ git log --oneline --graph --all
* 76eaf96 Add wishlist
| * e6f6181 fix: round totals
|/
* 7cb5715 Initial commit
$ git log --oneline origin/main..main
76eaf96 Add wishlist
```

1. **ahead 1, behind 1**: main diverged from origin.
2. My commit sits on main by mistake.
3. The teammate's fix is on origin/main.

TRY IT ACT

Move the commit and sync with the team

```
$ git branch backup
$ git branch feature/wishlist
$ git reset -q --hard origin/main
$ git switch -q feature/wishlist
$ git rebase -q main
$ git log --oneline --graph --all
* d12811d Add wishlist
* e6f6181 fix: round totals
| * 76eaf96 Add wishlist
|/
* 7cb5715 Initial commit
```

1. **backup** first: free insurance.
2. main = origin/main again.
3. The feature sits on top of the latest main.

STEP BY STEP CHEAT SHEET

"Oh no" → the fix

SITUATION	FIX
Committed to main by mistake	`git branch x; git reset --hard origin/main`
Wrong message / forgot a file	`git commit --amend`
Bad commit already shared	`git revert <id>`
Lost commits	`git reflog` → `git branch rescue <id>`
Push rejected	`git pull --rebase` → push
Need to switch NOW	`git stash -u` → … → `git stash pop`
Which commit broke it?	`git bisect run <test>`
Leaked secret	rotate → `git filter-repo` → prevent

COMMON MISTAKE RESCUE

Panic commands

✗ WRONG

```
rm -rf .git && git init      # "start fresh"
git push --force             # "make it work"
git reset --hard             # with unsaved work
```

These destroy history, teammates' work, or uncommitted changes, and they're the hardest mistakes to undo.

✓ RIGHT

```
git branch backup            # insurance first
git status -sb; git log --graph --all; git reflog
# then one small step at a time
```

Look first. Back up first. Small steps. Almost everything is recoverable if you don't panic.

IN REAL JOBS GIT PRO

You’re ready for real teams

Daily

```
git switch -c feature/x
git add -p && git commit
git pull --rebase && git push
```

Pull requests

```
small PRs, Conventional Commits,
rebase onto main, --force-with-lease
```

Releases

```
git tag -a v1.3.0 -m "Release 1.3.0"
git push origin v1.3.0
```

Emergencies

```
reflog · revert · bisect · stash
```

INTERVIEW SAY THIS IN THE INTERVIEW

“You committed to main instead of a feature branch. Fix it.”

- Keep the work: **git branch feature/x** at the current commit.
- Move main back: **git reset --hard origin/main** (only if not pushed!).
- Continue on **feature/x**; **git rebase main** if main moved.
- Already pushed to shared main? **git revert** instead, and re-apply on a branch.
- Prevent it: protect main on GitHub.

> **Follow-up they ask next:** ”How do you approach a repo in a weird state?” (look, find, plan, act, with a backup branch first).

RECAP REMEMBER THESE

Git Pro in 6 lines

1. Commit small and often; read your diff first.
2. Branch per task; PR + review + CI; protect main.
3. Rebase your own branch; revert shared commits.
4. fetch/pull --rebase; force only with --force-with-lease.
5. reflog, bisect, blame and -S solve the hard problems.
6. Hooks, .gitignore, .gitattributes, secret scanning from day one.

PRACTICE NOW

Do exercises/ex12_rescue_the_repo.py, then the final exam and the interview prep.